

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

Nombre del alumno:

ANGEL GASPAR REYES

8A

TSU EN TECNOLOGIAS DE LA
INFORMACION: DESARROLLO DE
SOFTWARE Y MULTIPLATAFORMA

TEHUACÁN, PUEBLA, 05 de Abril de 2026

1. ¿Qué es un *middleware*?

Un **middleware** es una función que se ejecuta **antes de que una petición llegue a tu ruta final**.

👉 Piensa en él como un guardia de seguridad:

- Revisa quién entra
- Valida credenciales
- Decide si pasa o no

Ejemplo en Express (Node.js):

```
const verificarToken = (req, res, next) => {  
  const token = req.headers['authorization'];  
  
  if (!token) {  
    return res.status(403).json({ mensaje: 'Acceso denegado' });  
  }  
  
  // aquí validarías el token  
  
  next(); // deja pasar a la siguiente función  
};
```

```
const verificarToken = (req, res, next) => {  
  const token = req.headers['authorization'];  
  
  if (!token) {  
    return res.status(403).json({ mensaje: 'Acceso denegado' });  
  }  
  
  // aquí validarías el token  
  next(); // deja pasar a la siguiente función  
};
```

2. ¿Cómo se protegen rutas en backend?

Se protegen usando middlewares en rutas sensibles.

Ejemplo:

```
app.get('/perfil', verificarToken, (req, res) => {  
  res.json({ mensaje: 'Bienvenido a tu perfil' });  
});
```

```
app.get('/perfil', verificarToken, (req, res) => {  
  res.json({ mensaje: 'Bienvenido a tu perfil' });  
});
```

Sin middleware → cualquiera entra

Con middleware → solo usuarios autorizados

3. Errores de seguridad comunes

- ❌ No validar tokens
- ❌ Confiar en datos del frontend
- ❌ No usar roles (todos tienen acceso a todo)
- ❌ Rutas accesibles directamente (ej: /admin)
- ❌ No expirar sesiones
- ❌ Mostrar errores sensibles del servidor

💡 Regla de oro:

“Nunca confíes en el cliente, valida TODO en el servidor.”

4. ¿Cómo se valida acceso por rol?

Aquí es donde subes de nivel

Primero defines roles:

// ejemplo de usuario

```
{  
  nombre: "Angel",  
  rol: "admin" // o "usuario"  
}
```

```
// ejemplo de usuario  
{  
  nombre: "Angel",  
  rol: "admin" // o "usuario"  
}
```

Middleware por rol:

```
const verificarRol = (rolPermitido) => {  
  return (req, res, next) => {  
    const usuario = req.usuario; // viene del token  
  
    if (usuario.rol !== rolPermitido) {  
      return res.status(403).json({ mensaje: 'No tienes permisos' });  
    }  
  
    next();  
  };  
};
```

```
const verificarRol = (rolPermitido) => {  
  return (req, res, next) => {  
    const usuario = req.usuario; // viene del token  
  
    if (usuario.rol !== rolPermitido) {  
      return res.status(403).json({ mensaje: 'No tienes permisos' });  
    }  
  
    next();  
  };  
};
```

Uso:

```
app.get('/admin', verificarToken, verificarRol('admin'), (req, res) => {  
  res.json({ mensaje: 'Panel de administrador' });  
});
```

```
app.get('/admin', verificarToken, verificarRol('admin'), (req, res) => {  
  res.json({ mensaje: 'Panel de administrador' });  
});
```

5. ¿Qué debes implementar?

✅ Protección de rutas

- /perfil → usuario autenticado
- /admin → solo admin

✅ Validación de rol

- Middleware que revise el rol desde el token

✅ Bloqueo de accesos indebidos

Prueba esto:

- Entrar sin token → ❌ Denegado
- Entrar con rol incorrecto → ❌ Denegado
- Entrar correctamente → ✅ Permitido

6. Tests de seguridad (básico pero funcional)

Puedes simular con Postman:

Caso 1:

- Sin token
Resultado esperado: **403**

Caso 2:

- Token válido pero rol incorrecto
Resultado: **403**

Caso 3:

- Token y rol correctos
Resultado: **200**

👉 No se trata solo de que funcione

👉 Se trata de que **nadie pueda romperlo fácilmente**